

Student

Five chapters of theory, design, and SQL mechanics. Now I want to **build something real** - from a blank database to a working dimensional warehouse with live data.

Claude • AI Assistant

This lab is the capstone. You will follow the four-step Kimball process, build the complete physical schema, load dimensions using SCD Type 2 MERGE, load a fact table with full DQ validation, handle a late-arriving dimension, correct a fact error with a negating entry, and run eight production analytics queries.

The domain: **Mango Mart** - a mid-sized Indian retail chain with stores across four regions. Everything you build in this lab has appeared in one of the five theory chapters.

The lab is the proof. Three parts:

1. **Lab Part 1 - Design and Build** (3 subsections): Apply the four-step process, build the Bus Matrix, build all DDL.
2. **Lab Part 2 - Load and Validate** (3 subsections): SCD Type 2 dimension load, fact load with DQ gate, late-arriving dimension and fact correction.
3. **Lab Part 3 - Analytics** (3 subsections): Sales performance queries, customer analytics (RFM + cohort), and inventory + cross-fact drill-across.

CONCEPT CARD

Mango Mart - the lab domain:

1. **47 flagship stores**, 22 express stores, 1 online channel - across North, South, East, West regions.
2. **500 active SKUs** across 5 categories: Grocery, Dairy, Electronics, Apparel, Stationery.
3. **2 million loyalty card customers** - segmented as New, Regular, Loyal, VIP.
4. **Business processes modeled**: Sales Transactions + Monthly Inventory Snapshot + Customer Returns.
5. **Data warehouse platform**: Standard SQL (PostgreSQL-compatible syntax throughout).

SUMMARY CARD

By the end of this lab you will have:

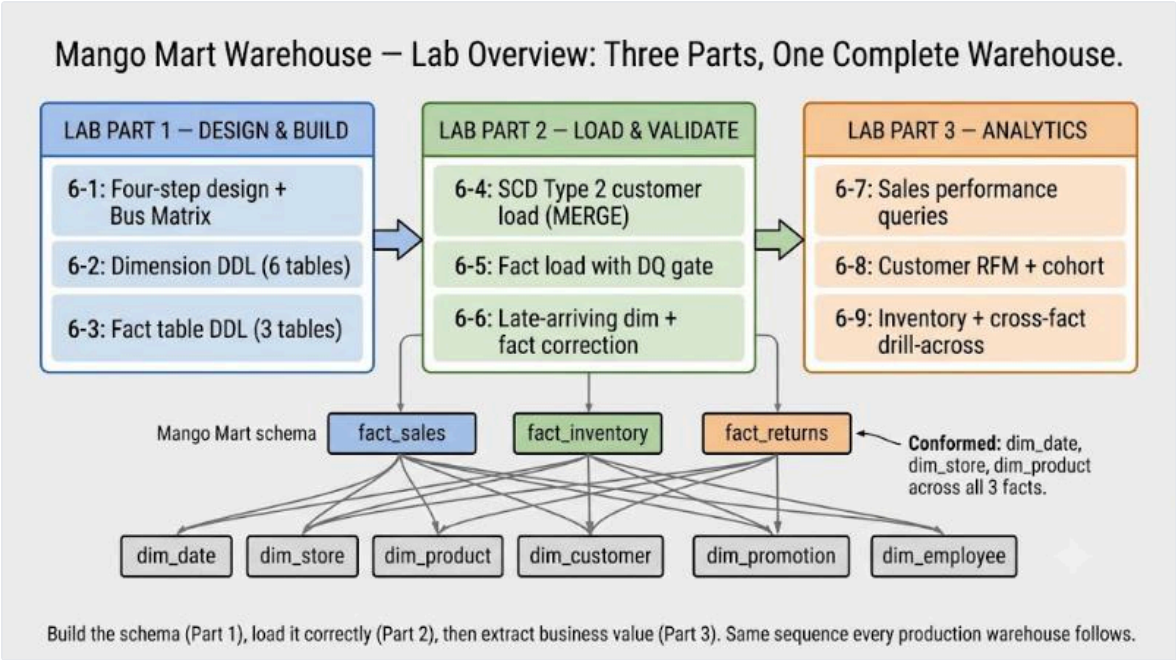
- A complete star schema with 6 dimensions and 3 fact tables built and loaded.
- A working SCD Type 2 customer dimension load using the two-step MERGE pattern.

- A DQ-gated fact load that rejects bad rows to a staging error table.
- A late-arriving dimension placeholder created and back-updated.
- A negating-entry fact correction applied and verified.
- Eight production analytics queries covering revenue, RFM, retention, inventory, and cross-fact analysis.

🌟 AI DEEP DIVE PROMPT

Warm-up (paste into ChatGPT or Claude): **"I am building a retail dimensional warehouse for a chain called Mango Mart. Business processes: sales transactions, monthly inventory snapshot, customer returns.**

Walk me through the four-step Kimball design process for each process: state the grain, list the dimensions, classify the measures. Then draw the Bus Matrix showing which dimensions are conformed across all three processes."



Mango Mart Warehouse - Lab Overview: Three Parts, One Complete Warehouse

Mango Mart Bus Matrix - Three Business Processes and Six Dimensions

Business Process	Grain	dim_date	dim_store	dim_product	dim_customer	dim_promotion
Sales Transactions	One POS line item	✓	✓	✓	✓ (nullable)	✓
Monthly Inventory	Product × store	✓	✓	✓		

Business Process	Grain	dim_date	dim_store	dim_product	dim_customer	dim_promo
Snapshot	× month-end					
Customer Returns	One returned line item	✓	✓	✓	✓	
Conformed?	-	YES (3✓)	YES (3✓)	YES (3✓)	YES (2✓)	N

```
sql – Lab setup – create the Mango Mart warehouse schema

-- Create the Mango Mart warehouse database
-- Run once before starting the lab.

CREATE DATABASE mango_mart_dw;
\c mango_mart_dw;

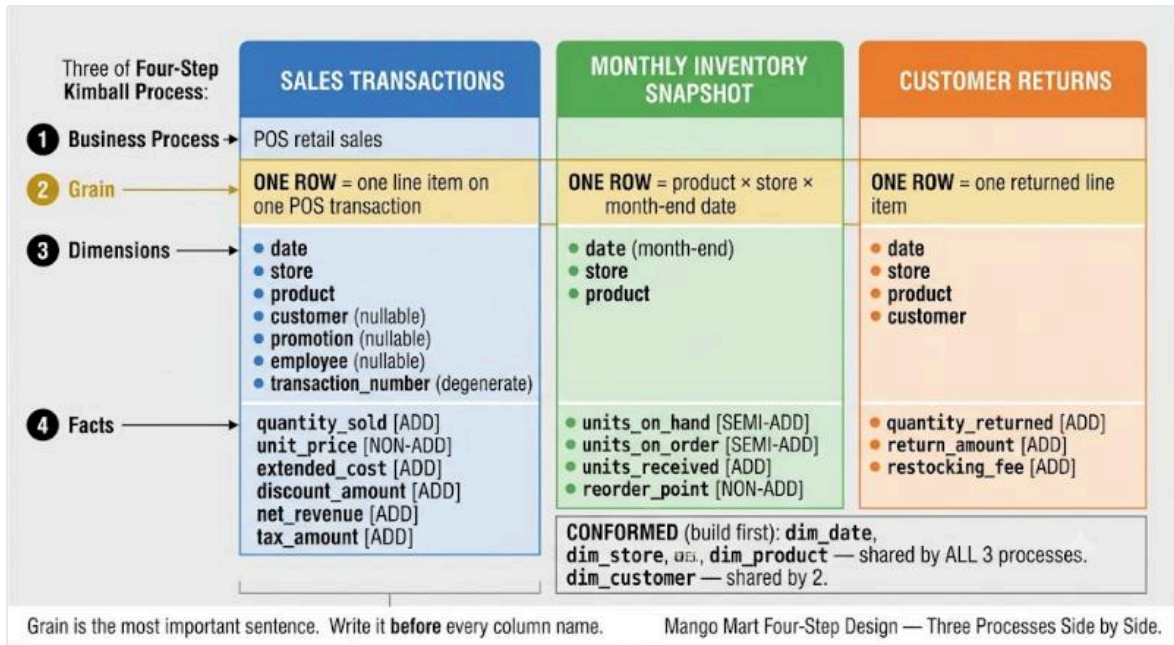
-- Surrogate key sequences (one per dimension)
CREATE SEQUENCE seq_date_key      START 20240101 INCREMENT 1;
CREATE SEQUENCE seq_store_key     START 1 INCREMENT 1;
CREATE SEQUENCE seq_product_key  START 1 INCREMENT 1;
CREATE SEQUENCE seq_customer_key START 1 INCREMENT 1;
CREATE SEQUENCE seq_promo_key    START 1 INCREMENT 1;
CREATE SEQUENCE seq_employee_key  START 1 INCREMENT 1;

-- Staging area for ETL (separate schema)
CREATE SCHEMA stg;
CREATE SCHEMA dw;

-- The lab builds all tables in the dw schema.
-- All sections below assume: SET search_path = dw, stg, public;
```

LAB PART 1-A — FOUR-STEP DESIGN AND BUS MATRIX

Claude • AI Assistant



Mango Mart Four-Step Design - Three Processes Side by Side

Student

Let's start at the very beginning - apply the four-step Kimball process to Mango Mart and build the Bus Matrix.

Claude • AI Assistant

Before writing a single CREATE TABLE, you need the Bus Matrix. The matrix tells you which dimensions to build first, which facts they unlock, and which dimensions are conformed across multiple processes.

This is the governance document that prevents the silo mistakes from Chapter 3.

CONCEPT CARD

Applying the four-step process to Mango Mart:

Process 1 - Sales Transactions:

1. Business process: point-of-sale retail transactions.
2. Grain: one row per line item on a POS transaction.
3. Dimensions: Date (when), Store (where), Product (what), Customer (who - nullable for anonymous), Promotion (if active), Employee (cashier - nullable for online).

4. Measures: *quantitysold* (ADD), *unitprice* (NON-ADD), *extendedcost* (ADD), *discountamount* (ADD), *netrevenue* (ADD), *taxamount* (ADD). Store *transaction_number* as degenerate dimension.

Process 2 - Monthly Inventory Snapshot:

1. Business process: stock-level tracking at month-end.
2. Grain: one row per product × store × calendar month-end date.
3. Dimensions: Date (month-end only), Store, Product.
4. Measures: *unitsonhand* (SEMI-ADD), *unitsonorder* (SEMI-ADD), *unitsreceivedinperiod* (ADD), *reorderpoint* (NON-ADD).

Process 3 - Customer Returns:

1. Business process: returned merchandise processing.
2. Grain: one row per returned line item.
3. Dimensions: Date, Store, Product, Customer.
4. Measures: *quantityreturned* (ADD), *returnamount* (ADD), *restocking_fee* (ADD).

CONCEPT CARD

Reading the Bus Matrix:

1. **dim_date** - 3 checkmarks: the most conformed dimension. Build it first.
2. **dim_store** - 3 checkmarks: build second.
3. **dim_product** - 3 checkmarks: build third.
4. **dim_customer** - 2 checkmarks (Sales + Returns): conformed across those two.
5. **dim_promotion** - 1 checkmark (Sales only): process-specific, no conformance needed.
6. **dim_employee** - 1 checkmark (Sales only): process-specific.
7. **Build sequence:** *dimdate* → *dimstore* → *dimproduct* → *dimcustomer* → *dimpromotion* → *dimemployee* → *factsales* → *factinventory* → *fact_returns*.

EXAMPLE CARD

What the Bus Matrix tells your team:

1. Any developer building the Returns pipeline knows to use the same **dim_product** and **dim_customer** keys as the Sales pipeline.
2. The inventory pipeline does NOT need to JOIN to **dim_customer** - no customer data in inventory.
3. If a new 'Customer Satisfaction Survey' business process is proposed, the Bus Matrix review asks: does it use **dim_customer** ? Same keys? This is the governance gate.
4. The drill-across query (Section 6-9) works ONLY because all three facts share **dim_date** , **dim_store** , and **dim_product** with identical surrogate keys.

🔗 ANALOGY

The Bus Matrix is like the **wiring diagram of a building before construction starts**. The electrician (ETL developer) for the Sales circuit and the electrician for the Inventory circuit both look at the same diagram.

They know which outlets are shared (conformed dimensions) and which are dedicated to one circuit (process-specific). Wiring without this diagram produces a building where the lights in one room are on the circuit breaker for another.

⚠️ WARNING

Do not start writing DDL before the Bus Matrix is complete and reviewed. A dimension built for one process that doesn't conform to the master definition will require a costly key migration later.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Each team builds their own date dimension with their own surrogate key sequence. The inventory team's `date_key = 3` means March 31, 2024. The sales team's `date_key = 3` means January 3, 2024. The drill-across query joins on `date_key` and produces nonsense.

Correct Way: One master `dim_date` built by the warehouse team, shared by all three processes. `date_key = 20240331` (YYYYMMDD integer) means March 31, 2024 everywhere, always.

EXAM TIP CARD

Interview classic: "How do you sequence the build of a dimensional warehouse when multiple teams are working in parallel?" Winning answer: "Bus Matrix first. Identify the most-conformed dimensions - those with the most checkmarks across business process rows.

Build those dimensions first; they unlock the most fact tables. Agree on surrogate key sequences for each shared dimension before any team starts coding.

Fact tables are built last, after all their dimension dependencies are complete."

🔥 EXERCISE

Add a fourth business process to the Mango Mart Bus Matrix: 'Promotional Coverage' (factless - tracks which products were promoted in which stores on which dates). Grain: one row per date × store × product × promotion.

Which dimensions does it add checkmarks to? Does it change any conformance status?

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: **"For a grocery chain with four business processes (sales, inventory, supplier deliveries, loyalty point redemption), build the complete Bus Matrix. Identify which three dimensions to build first and why.**

Write the grain statement for each business process in one sentence."

LAB PART 1-B — DIMENSION DDL

Claude • AI Assistant

Mango Mart Dimension DDL — SCD Type Decision per Dimension			
Dimension name	SCD Type	Tracked attributes (Type 2 only)	Key design note
dim_date	Type 0 (pre-built)	n/a	YYYYMMDD int key, April fiscal year, relative_period_month column.
dim_store	Type 1	n/a	Hierarchy flattened: store → city → state → region. No history needed.
dim_product	Type 2	unit_cost	category = Type 1 (overwrite). product_name = Type 1. unit_cost = Type 2. is_current + row dates.
dim_customer	Type 2	loyalty_tier, city	is_placeholder flag for late-arriving handling. row_start, row_end, is_current. Unknown Customer row (key = -1).
dim_promotion	Type 0	n/a	Promotion definition frozen at creation. Never updated.
dim_employee	Type 2	store_assignment, job_role	employee_key = -1 for online (no cashier) transactions.
Build order: dim_date → dim_store → dim_product → dim_customer → dim_promotion → dim_employee. Fact tables built last.			
Apply Type 2 only where historical value has analytical meaning. Every Type 2 column generates rows — cost of accuracy.			

Mango Mart Dimension DDL - SCD Type Decision per Dimension

sql – All six Mango Mart dimension tables – complete DDL

```
-- =====
-- DIM_DATE: pre-built calendar, April fiscal year
-- =====

CREATE TABLE dw.dim_date (
    date_key          INT          PRIMARY KEY, -- YYYYMMDD
    calendar_date     DATE         NOT NULL UNIQUE,
    day_name          VARCHAR(10)  NOT NULL,
    day_of_week_num   TINYINT      NOT NULL,
    is_weekend        TINYINT      NOT NULL DEFAULT 0,
    month_number       TINYINT      NOT NULL,
    month_name        VARCHAR(10)  NOT NULL,
```

```

quarter_number      TINYINT      NOT NULL,
year_number          SMALLINT     NOT NULL,
fiscal_year          SMALLINT     NOT NULL,  -- Apr-to-Mar
fiscal_quarter       TINYINT      NOT NULL,
fiscal_month         TINYINT      NOT NULL,  -- April=1, March=12
is_holiday           TINYINT      NOT NULL DEFAULT 0,
is_business_day      TINYINT      NOT NULL DEFAULT 1,
relative_period_month INT         NOT NULL DEFAULT 0  -- 0=current month
);

-- =====
-- DIM_STORE: Type 1, hierarchy flattened
-- =====

CREATE TABLE dw.dim_store (
    store_key      INT          PRIMARY KEY DEFAULT NEXTVAL('seq_store_key'),
    store_code     VARCHAR(10) NOT NULL UNIQUE,
    store_name     VARCHAR(200),
    city           VARCHAR(100),
    state          VARCHAR(50),
    region         VARCHAR(20),  -- North / South / East / West / Online
    store_format   VARCHAR(15),  -- Flagship / Express / Online
    open_date      DATE,
    is_active      TINYINT      NOT NULL DEFAULT 1
);

-- Unknown Store placeholder
INSERT INTO dw.dim_store VALUES (-1,'UNKNOWN','Unknown Store',
    'Unknown','Unknown','Unknown','Unknown',NULL,1);

-- =====
-- DIM_PRODUCT: Type 2 on unit_cost; Type 1 on category/name
-- =====

CREATE TABLE dw.dim_product (
    product_key      INT          PRIMARY KEY DEFAULT
NEXTVAL('seq_product_key'),
    product_code     VARCHAR(20) NOT NULL,
    product_name     VARCHAR(200) NOT NULL,
    brand            VARCHAR(100),
    subcategory_name VARCHAR(100),
    category_name    VARCHAR(100),
    unit_cost        DECIMAL(10,2),  -- TRACKED (Type 2)
    unit_of_measure  VARCHAR(20),
    is_active        TINYINT      NOT NULL DEFAULT 1,
    row_start_date   DATE          NOT NULL DEFAULT CURRENT_DATE,
    row_end_date     DATE          NOT NULL DEFAULT '9999-12-31',
    is_current       TINYINT      NOT NULL DEFAULT 1
);

```

```

-- =====
-- DIM_CUSTOMER: Type 2 on loyalty_tier + city
-- =====

CREATE TABLE dw.dim_customer (
    customer_key    INT                PRIMARY KEY DEFAULT
NEXTVAL('seq_customer_key'),
    customer_id     VARCHAR(20) NOT NULL,    -- natural key
    customer_name   VARCHAR(200) NOT NULL,
    email           VARCHAR(200),
    city            VARCHAR(100),            -- TRACKED (Type 2)
    loyalty_tier    VARCHAR(15),            -- TRACKED (Type 2)
    acquisition_ch  VARCHAR(50),
    first_purchase_dt DATE,
    row_start_date  DATE                NOT NULL DEFAULT CURRENT_DATE,
    row_end_date    DATE                NOT NULL DEFAULT '9999-12-31',
    is_current      TINYINT             NOT NULL DEFAULT 1,
    is_placeholder  TINYINT             NOT NULL DEFAULT 0
);

-- Unknown Customer placeholder (for anonymous transactions)
INSERT INTO dw.dim_customer
    (customer_key, customer_id, customer_name, email, city,
     loyalty_tier, acquisition_ch, is_current, is_placeholder)
VALUES (-1, 'UNKNOWN', 'Unknown Customer', NULL, 'Unknown',
        'Unknown', 'Unknown', 1, 0);

-- =====
-- DIM_PROMOTION: Type 0 (never changes after creation)
-- =====

CREATE TABLE dw.dim_promotion (
    promotion_key    INT                PRIMARY KEY DEFAULT
NEXTVAL('seq_promo_key'),
    promotion_code   VARCHAR(20) NOT NULL UNIQUE,
    promotion_name   VARCHAR(200),
    promotion_type   VARCHAR(50), -- 'Percentage Off', 'Buy2Get1', 'Bundle'
    discount_pct     DECIMAL(5,2),
    campaign         VARCHAR(100),
    start_date       DATE,
    end_date         DATE
);

-- =====
-- DIM_EMPLOYEE: Type 2 on store_assignment + job_role
-- =====

CREATE TABLE dw.dim_employee (
    employee_key    INT                PRIMARY KEY DEFAULT

```

```

NEXTVAL('seq_employee_key'),
    employee_id    VARCHAR(15) NOT NULL,
    employee_name  VARCHAR(200) NOT NULL,
    job_role       VARCHAR(50),    -- TRACKED (Type 2)
    department     VARCHAR(50),
    store_key      INT,
    hire_date      DATE,
    row_start_date DATE          NOT NULL DEFAULT CURRENT_DATE,
    row_end_date   DATE          NOT NULL DEFAULT '9999-12-31',
    is_current     TINYINT       NOT NULL DEFAULT 1
);
-- Online/no-cashier placeholder
INSERT INTO dw.dim_employee
    (employee_key, employee_id, employee_name, job_role, department)
VALUES (-1, 'ONLINE', 'Online Channel', 'N/A', 'Digital');

```

Student

Build all six dimensions with production-quality DDL.

Claude • AI Assistant

Six dimension tables. The three conformed dimensions (date, store, product) are built first.

Then the two process-specific dimensions (promotion, employee) and the SCD Type 2 customer dimension. Every dimension has the standard pattern: surrogate key, natural key, descriptive attributes, and SCD columns where needed.

CONCEPT CARD

Dimension design decisions for Mango Mart:

1. **dim_date** - pre-built for 5 years (2023–2027). YYYYMMDD integer key. Fiscal year: April-to-March. **relative_period_month** updated monthly by ETL.
2. **dim_store** - Type 1 (store attributes like address and format rarely change and history not needed). Surrogate key, store_code natural key, all hierarchy levels flattened (store → city → state → region).
3. **dim_product** - Type 2 (*unitcost changes → need to know cost at time of transaction*). Surrogate key, productcode natural key, full hierarchy flattened (product → subcategory → category), **is_current** + row dates.
4. **dim_customer** - Type 2 (*loyaltytier and city change → need historical accuracy*). The full SCD Type 2 treatment: surrogate key, customerid natural key, rowstart/end dates,

`iscurrent`, `is_placeholder` flag.

5. **dim_promotion** - Type 0 (promotion definitions never change retroactively). Simple lookup table.
6. **dim_employee** - Type 2 (role and store assignment can change). Surrogate key, `employee_id` natural key, SCD Type 2 columns.

CONCEPT CARD

The SCD decision matrix for Mango Mart:

1. `customer.loyalty_tier` - Type 2. 'What tier was the customer when they bought this?' matters for historical reporting.
2. `customer.city` - Type 2. Shipping/store analytics need to know where the customer lived at purchase time.
3. `store.address` - Type 1. If a store moves, we don't care about historical address for sales analytics.
4. `product.unit_cost` - Type 2. Gross margin calculations need the cost at the time of the transaction.
5. `product.category` - Type 1. Category reclassification applies retroactively (we always want products in their current category).
6. `promotion.discount_pct` - Type 0. The historical promotion definition must never change.

⚠ WARNING

Not every changing attribute needs Type 2. Type 2 generates rows - millions of customers with monthly attribute changes produce billions of dimension rows. Apply Type 2 only where 'what was the value AT THE TIME of the transaction' is a genuine business question.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: Applying Type 2 to `customer.phone_number`. Nobody ever asks 'what phone number did this customer have when they made this purchase?'

Correct Way: Type 1 on `phone_number` (just overwrite). Reserve Type 2 for attributes where the historical value has analytical meaning.

EXAM TIP CARD

Interview classic: "For a customer dimension, which attributes should be Type 1 and which Type 2?" Winning answer: "Type 2 when you need 'what was the value at the time of the transaction' - e.g., loyalty tier, city, income segment. Type 1 when you just want the current value - e.g., phone number, email, preferred language.

Type 0 for identifiers that must never change retroactively."*

🔥 EXERCISE

For each attribute, decide Type 0, 1, or 2 and justify: (a) store format (flagship/express/online), (b) product brand, (c) employee salary grade, (d) customer's preferred language, (e) promotion campaign name.

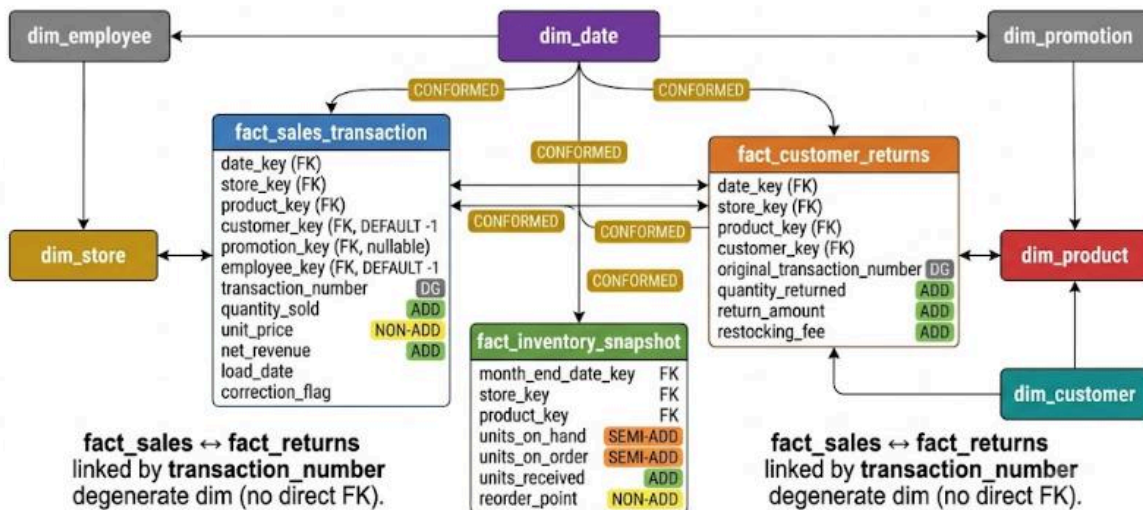
AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "For a retail employee dimension with attributes: *employee_id*, *employee_name*, *store_assignment*, *job_role*, *department*, *hire_date*, *salary_grade*, *emergency_contact* - classify each attribute as SCD Type 0, 1, or 2 with a one-sentence justification for each. Then write the complete dimension DDL."

LAB PART 1-C — FACT TABLE DDL

Claude • AI Assistant

Mango Mart Star Schema — Three Facts, Six Dimensions, FK Connections



Six columns in fact = six checkmarks in the Bus Matrix. Every FK traces back to a dimension row.

Mango Mart Star Schema - Three Facts, Six Dimensions, FK Connections

sql – Three Mango Mart fact tables – complete DDL with correction metadata

```
-- =====
-- FACT_SALES_TRANSACTION
-- Grain: one row per line item on a POS transaction
-- =====

CREATE TABLE dw.fact_sales_transaction (
    date_key          INT          NOT NULL,    -- FK → dim_date
```

```

        store_key          INT          NOT NULL,      -- FK → dim_store
        product_key        INT          NOT NULL,      -- FK → dim_product
(is_current=1 at load)
        customer_key       INT          NOT NULL DEFAULT -1,  -- FK →
dim_customer (-1=anonymous)
        promotion_key      INT,          -- FK → dim_promotion
(NULL=no promotion)
        employee_key       INT          NOT NULL DEFAULT -1,  -- FK →
dim_employee (-1=online)
        transaction_number VARCHAR(25) NOT NULL,      -- DEGENERATE DIMENSION
        quantity_sold      INT          NOT NULL,
        unit_price          DECIMAL(10,2),          -- NON-additive: NEVER SUM
        extended_cost      DECIMAL(12,2),
        discount_amount    DECIMAL(10,2) DEFAULT 0,
        net_revenue        DECIMAL(12,2) NOT NULL,
        tax_amount         DECIMAL(10,2) DEFAULT 0,
        -- Correction metadata (added at creation – not retrofitted)
        load_date          DATE          NOT NULL DEFAULT CURRENT_DATE,
        correction_flag     TINYINT      NOT NULL DEFAULT 0,
        correction_reason   VARCHAR(50)
    );
CREATE INDEX idx_fst_date    ON dw.fact_sales_transaction(date_key);
CREATE INDEX idx_fst_product ON dw.fact_sales_transaction(product_key,
date_key);
CREATE INDEX idx_fst_store   ON dw.fact_sales_transaction(store_key,
date_key);
CREATE INDEX idx_fst_cust    ON dw.fact_sales_transaction(customer_key);

-- =====
-- FACT_INVENTORY_SNAPSHOT
-- Grain: one row per product × store × month-end date
-- =====
CREATE TABLE dw.fact_inventory_snapshot (
    month_end_date_key INT          NOT NULL,      -- FK → dim_date (month-
end rows only)
    store_key          INT          NOT NULL,      -- FK → dim_store
    product_key        INT          NOT NULL,      -- FK → dim_product
    units_on_hand      INT          NOT NULL,      -- SEMI-ADDITIVE (sum by
store/product, not time)
    units_on_order     INT          NOT NULL DEFAULT 0,  -- SEMI-ADDITIVE
    units_received     INT          NOT NULL DEFAULT 0,  -- additive
(period flow)
    reorder_point      INT,          -- NON-additive (policy
threshold)
    load_date          DATE          NOT NULL DEFAULT CURRENT_DATE,
    -- Composite PK prevents duplicate month-end snapshots

```

```

PRIMARY KEY (month_end_date_key, store_key, product_key)
);

-- =====
-- FACT_CUSTOMER_RETURNS
-- Grain: one row per returned line item
-- =====

CREATE TABLE dw.fact_customer_returns (
    date_key            INT            NOT NULL,
    store_key           INT            NOT NULL,
    product_key         INT            NOT NULL,
    customer_key        INT            NOT NULL, -- returns are always
linked to a customer
    original_transaction_number VARCHAR(25),    -- DEGENERATE: links
back to the sale
    return_reason_code  VARCHAR(20),
    quantity_returned   INT            NOT NULL,
    return_amount       DECIMAL(12,2) NOT NULL,
    restocking_fee      DECIMAL(10,2) DEFAULT 0,
    load_date           DATE           NOT NULL DEFAULT CURRENT_DATE,
    correction_flag     TINYINT        NOT NULL DEFAULT 0
);

CREATE INDEX idx_fcr_date    ON dw.fact_customer_returns(date_key);
CREATE INDEX idx_fcr_product ON dw.fact_customer_returns(product_key);

```

Student

Now build the three fact tables.

Claude • AI Assistant

Three fact tables at three different grains. The DDL encodes the Bus Matrix: each FK column corresponds to a checkmark in the matrix. Correction metadata (**load_date** , **correction_flag**) is added from day one - retrofitting it later is painful.

CONCEPT CARD

Fact table design decisions:

1. **fact_sales_transaction** - the busiest table. NOT NULL FKs everywhere except customer (-1 = anonymous), promotion (NULL = no active promo), employee (-1 = online). **transaction_number** degenerate dimension as VARCHAR - not a FK.

2. **fact_inventory_snapshot** - composite PK on (monthenddatekey, storekey, productkey) - uniqueness enforced at the database level. `unitsonhand` and `unitson\_order` are semi-additive - documented in comments.
3. **fact_customer_returns** - links back to the original sale via `original_transaction_number` (degenerate dimension from the sales receipt). No direct FK between the two fact tables - linked through the conformed degenerate dimension.

CONCEPT CARD

Correction metadata on every fact table:

1. `load_date DATE` - which ETL run inserted this row.
2. `correction_flag TINYINT DEFAULT 0` - 0 for original rows, 1 for negation or correction rows.
3. `correction_reason VARCHAR(50)` - NULL for originals; 'DECIMALERROR', 'DUPLICATEBATCH', 'WRONG_STORE' for corrections.
4. Adding these at creation time costs one integer and one tinyint per row - negligible. Adding them after the fact requires an ALTER TABLE on a billion-row fact table - painful.

⚠ WARNING

The composite PK on `fact_inventory_snapshot` prevents duplicate month-end snapshots for the same product × store - a common ETL idempotency bug. Without it, a re-run of the monthly snapshot ETL doubles all inventory figures silently.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Making `promotion_key` NOT NULL in `fact_sales_transaction`. Any transaction without a promotion cannot be loaded, silently excluding the majority of sales.

Correct Way: `promotion_key INT` (nullable). NULL means no promotion was active on this line item.

EXAM TIP CARD

Interview classic: "Should fact table foreign keys be nullable or NOT NULL?" Winning answer: "Depends on the business reality. For dimensions that always have a value (date, store, product) - NOT NULL with an Unknown surrogate.

For optional dimensions (customer if anonymous, promotion if none active) - either nullable or use an 'N/A' surrogate row. Never use NULL for dimensions that must always be present; never force NOT NULL for genuinely optional dimensions."

🔥 EXERCISE

Design the `correctionflag` report query: for `factsalestransaction`, write SQL that shows the net revenue per day (SUM of all rows including corrections) alongside the pre-correction

✦ AI DEEP DIVE PROMPT

How does the correction metadata interact with columnar compression?"*

LAB PART 2-A — SCD TYPE 2 CUSTOMER DIMENSION LOAD

AFTER DAY 1 LOAD — 3 rows, all current						
customer_key	customer_id	city	loyalty_tier	row_start	row_end	is_current
1001	C001	Chennai	Bronze	2024-01-01	9999-12-31	1
1002	C002	Delhi	Bronze	2024-01-01	9999-12-31	1
1003	C003	Pune	Gold	2024-01-01	9999-12-31	1

AFTER DAY 30 LOAD — 5 rows						
customer_key	customer_id	city	loyalty_tier	row_start	row_end	is_current
1001	C001	Chennai	Bronze	2024-01-01	2024-01-29	0
		EXPIRED — city changed				
1002	C002	Delhi	Bronze	2024-01-01	2024-01-29	0
		EXPIRED — tier changed				
1003	C003	Pune	Gold	2024-01-01	9999-12-31	1
		UNCHANGED — no action				
NEW 1004	C001	Mumbai	Bronze	2024-01-30	9999-12-31	1
		NEW VERSION — city=Mumbai				
NEW 1005	C002	Delhi	Silver	2024-01-30	9999-12-31	1
		NEW VERSION — tier=Silver				

SCD Type 2 Two-Step Load — dim_customer Before and After Day 30.

SCD Type 2 Two-Step Load - dim_customer Before and After Day 30

sql – SCD Type 2 – two-step MERGE for Mango Mart dim_customer

```
-- =====
-- STAGING TABLE: incoming customer records from CRM
-- =====

CREATE TABLE stg.stg_customer (
    customer_id    VARCHAR(20),
    customer_name  VARCHAR(200),
    email          VARCHAR(200),
```

```

    city          VARCHAR(100),
    loyalty_tier   VARCHAR(15),
    acquisition_ch VARCHAR(50)
);

-- Load Day 30 staging data
INSERT INTO stg.stg_customer VALUES
    ('C001','Priya Nair', 'priya@email.com', 'Mumbai', 'Bronze',
    'Online'), -- city changed
    ('C002','Rahul Sharma', 'rahul@email.com', 'Delhi', 'Silver', 'In-
    Store'),-- tier changed
    ('C003','Ananya Iyer', 'ananya@email.com', 'Pune', 'Gold',
    'Referral');-- unchanged

-- =====
-- STEP 1: MERGE – expire changed rows
-- (WHEN NOT MATCHED also inserts brand-new customers)
-- =====
MERGE INTO dw.dim_customer tgt
USING (
    SELECT customer_id, customer_name, email, city, loyalty_tier,
    acquisition_ch
    FROM stg.stg_customer
) src
ON (tgt.customer_id = src.customer_id AND tgt.is_current = 1)

-- Case A: Brand-new customer (no match in dim)
WHEN NOT MATCHED THEN
    INSERT (customer_id, customer_name, email, city, loyalty_tier,
    acquisition_ch, row_start_date, row_end_date, is_current)
    VALUES (src.customer_id, src.customer_name, src.email, src.city,
    src.loyalty_tier, src.acquisition_ch,
    CURRENT_DATE, '9999-12-31', 1)

-- Case B: Existing customer with changed tracked attributes – EXPIRE
WHEN MATCHED
    AND (tgt.city <> src.city
    OR tgt.loyalty_tier <> src.loyalty_tier)
THEN
    UPDATE SET
        tgt.row_end_date = CURRENT_DATE - INTERVAL '1 day',
        tgt.is_current = 0
-- Case C: Unchanged – no action (implicit)
;

-- =====

```

```
-- STEP 2: INSERT new versions for all just-expired rows
-- =====
INSERT INTO dw.dim_customer
    (customer_id, customer_name, email, city, loyalty_tier,
     acquisition_ch, row_start_date, row_end_date, is_current)
SELECT
    src.customer_id, src.customer_name, src.email,
    src.city, src.loyalty_tier, src.acquisition_ch,
    CURRENT_DATE, '9999-12-31', 1
FROM   stg.stg_customer src
JOIN   dw.dim_customer tgt
    ON  tgt.customer_id = src.customer_id
    AND tgt.is_current  = 0
    AND tgt.row_end_date = CURRENT_DATE - INTERVAL '1 day';

-- =====
-- CONSISTENCY CHECK: run after every dimension load
-- =====
SELECT customer_id, customer_key, row_end_date, is_current,
       'MISMATCH' AS issue
FROM   dw.dim_customer
WHERE  (is_current = 1 AND row_end_date <> '9999-12-31')
    OR (is_current = 0 AND row_end_date = '9999-12-31');
-- Expected: 0 rows. Any rows = load failure requiring investigation.
```

Student

The schema is built. Now load the customer dimension using the SCD Type 2 MERGE pattern from Chapter 4.

Claude • AI Assistant

This is the most important ETL pattern in dimensional modeling. The two-step MERGE: Step 1 expires changed rows (UPDATE current rows that differ), Step 2 inserts new versions for all changed and new records.

We'll trace it through three customer records including one city change and one loyalty tier upgrade.

CONCEPT CARD

The Mango Mart customer load scenario:

1. **Day 1 load** - 3 customers arrive from the CRM system. All are new.
2. **Day 30 load** - same 3 customers return. Customer C001 has moved from Chennai to Mumbai. Customer C002 has upgraded from Bronze to Silver tier. Customer C003 is unchanged.
3. **Expected outcome after Day 30:** 5 rows in dim_customer - 3 original rows (expired), 2 new versions (updated attributes), 1 still-current row (C003 unchanged).

CONCEPT CARD

The MERGE walkthrough:

1. **Step 1 - MERGE to expire:** for each incoming source row, JOIN to dimcustomer on customerid WHERE iscurrent = 1. If tracked attributes differ → UPDATE: rowenddate = CURRENTDATE - 1, is_current = 0. If no match → INSERT (new record). If no change → no action.
2. **Step 2 - INSERT new versions:** for all rows just expired (rowenddate = yesterday, is_current = 0 AND the source still has the updated values), INSERT new current rows with the new attributes.
3. **Consistency check** after every load: `SELECT * FROM dim_customer WHERE (is_current=1 AND row_end_date <> '9999-12-31') OR (is_current=0 AND row_end_date = '9999-12-31')` → must return 0 rows.

EXAMPLE CARD

Row-by-row walkthrough for Day 30 load:

1. **C001** (city changed Chennai→Mumbai): Step 1 - UPDATE rowkey=1001: rowend=2024-01-29, iscurrent=0. Step 2 - INSERT rowkey=1004: city=Mumbai, rowstart=2024-01-30, rowend=9999-12-31, is_current=1.
2. **C002** (tier changed Bronze→Silver): Step 1 - UPDATE rowkey=1002: rowend=2024-01-29, iscurrent=0. Step 2 - INSERT rowkey=1005: loyaltytier=Silver, rowstart=2024-01-30, is_current=1.
3. **C003** (no change): Step 1 - MATCHED AND unchanged → no action. Step 2 - no new row.
4. **Fact rows** loaded Jan 1–29 carry customerkey=1001 (Chennai version). Fact rows from Jan 30 onward carry customerkey=1004 (Mumbai version). Historical accuracy is embedded in the FK itself.

🔗 ANALOGY

The SCD Type 2 MERGE is like **updating a student's record file at a university**. When a student changes their hostel address, the registrar doesn't erase the old address.

They mark it as 'superseded as of [date]' and create a new record with the current address. Any exam result from before the move is filed under the old address record.

The student's academic history is correct because each record has a validity period.

⚠️ WARNING

Run the consistency check after EVERY dimension load. A partial-write failure (network timeout mid-MERGE) can leave rows where `is_current=1` AND `row_end_date` is not `9999-12-31` - a corrupted dimension state that will silently mis-route ETL lookups.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Doing both the expiry and the new version INSERT in a single MERGE statement. Most SQL dialects don't allow INSERT in a WHEN MATCHED clause. The first insert corrupts the match logic for subsequent rows in the same batch.

Correct Way: Two-step: Step 1 MERGE (UPDATE to expire). Step 2 separate INSERT (new versions). Always in that order.

EXAM TIP CARD

Interview classic: "Walk me through loading a customer dimension update where 30% of customers changed their loyalty tier." Winning answer: "Two-step SCD Type 2. Step 1: MERGE staging to `dimcustomer` on `customerid` WHERE `is_current=1`.

WHEN MATCHED AND `loyaltytier` changed: `UPDATE rowend=yesterday, is_current=0`.
WHEN NOT MATCHED: INSERT new row.

Step 2: INSERT new current rows for all customers just expired in Step 1 (join to staging to get the new `loyalty_tier` value). Post-load: consistency check.

Result: 30% of customers now have two rows - one expired, one current."

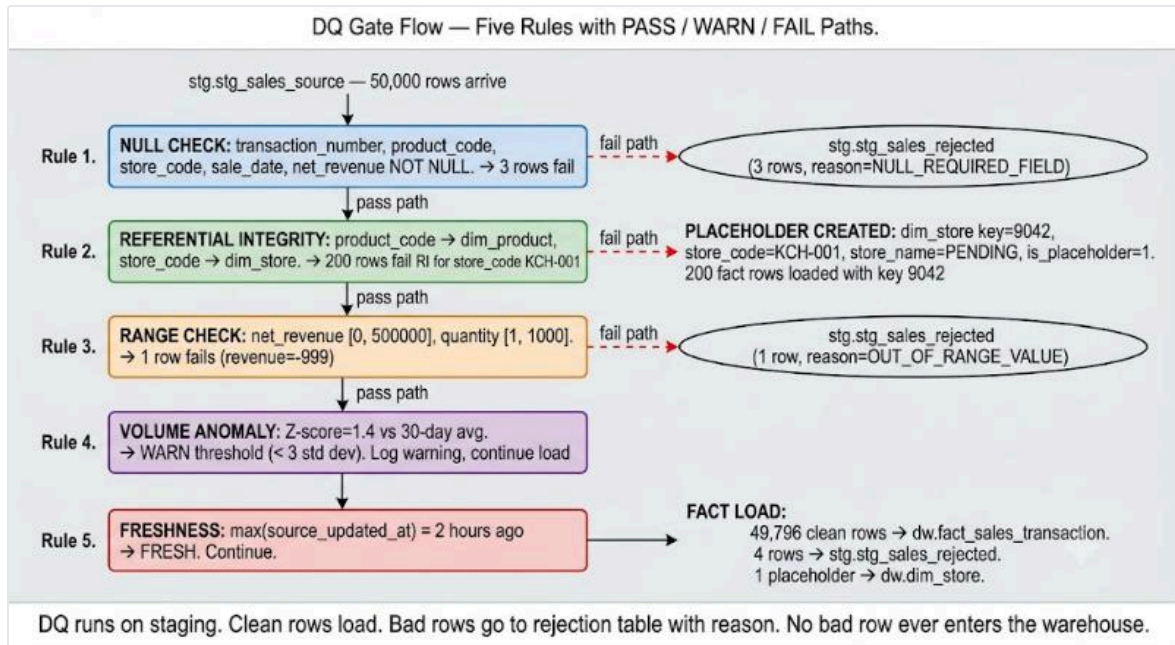
🔥 EXERCISE

After the Day 30 load described above, write the point-in-time query: 'What was Customer C001's loyalty tier on January 15?' Use the `row_start_date` and `row_end_date` columns to retrieve the correct historical version.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Write the complete two-step SCD Type 2 MERGE for a customer dimension in PostgreSQL. Include: the staging table DDL, Step 1 MERGE (expire changed rows), Step 2 INSERT (new versions), the consistency check query, and a point-in-time lookup query for a specific historical date."

LAB PART 2-B — FACT LOAD WITH DQ GATE AND LATE-ARRIVING DIMENSION



DQ Gate Flow - Five Rules with PASS / WARN / FAIL Paths

sql – DQ gate, placeholder creation, and fact load

```
-- =====
-- REJECTION TABLE
-- =====

CREATE TABLE stg.stg_sales_rejected AS
SELECT *, CURRENT_DATE AS reject_date,
       ''::VARCHAR(100) AS reject_reason
FROM   stg.stg_sales_source WHERE 1=0;

-- =====
-- RULE 1: NULL CHECK – reject rows with any NULL in required columns
-- =====

INSERT INTO stg.stg_sales_rejected
SELECT *, CURRENT_DATE, 'NULL_REQUIRED_FIELD'
FROM   stg.stg_sales_source
WHERE  transaction_number IS NULL
       OR product_code    IS NULL
       OR store_code      IS NULL
       OR sale_date       IS NULL
       OR net_revenue     IS NULL;

-- =====
-- RULE 2: RI CHECK – store_code not in dim_store
-- Create placeholder for unresolved store codes
-- =====
```

```

INSERT INTO dw.dim_store
    (store_key, store_code, store_name, city, state, region,
     store_format, is_active)
SELECT
    NEXTVAL('seq_store_key'),
    src.store_code,
    'PENDING - ' || src.store_code,
    'Pending', 'Pending', 'Pending', 'Pending', 1
FROM (
    SELECT DISTINCT store_code FROM stg.stg_sales_source src
    WHERE NOT EXISTS (
        SELECT 1 FROM dw.dim_store ds WHERE ds.store_code = src.store_code
    )
) unmatched;
-- In production: also add is_placeholder=1 flag and alert the team

-- =====
-- RULE 3: RANGE CHECK – reject out-of-range measures
-- =====

INSERT INTO stg.stg_sales_rejected
SELECT *, CURRENT_DATE, 'OUT_OF_RANGE'
FROM   stg.stg_sales_source
WHERE  net_revenue < 0 OR net_revenue > 500000
      OR quantity_sold < 1 OR quantity_sold > 1000;

-- =====
-- LOAD FACT: only clean rows (not in rejection table)
-- =====

INSERT INTO dw.fact_sales_transaction
    (date_key, store_key, product_key, customer_key,
     promotion_key, employee_key, transaction_number,
     quantity_sold, unit_price, net_revenue, tax_amount,
     load_date, correction_flag)
SELECT
    d.date_key,
    s.store_key,
    p.product_key,
    COALESCE(c.customer_key, -1),
    pr.promotion_key,           -- NULL if no promotion
    COALESCE(e.employee_key, -1),
    src.transaction_number,
    src.quantity_sold,
    src.unit_price,
    src.net_revenue,
    src.tax_amount,
    CURRENT_DATE, 0

```

```

FROM    stg.stg_sales_source src
JOIN    dw.dim_date           d  ON d.calendar_date = src.sale_date
JOIN    dw.dim_store          s  ON s.store_code    = src.store_code
JOIN    dw.dim_product        p  ON p.product_code  = src.product_code
                                AND p.is_current    = 1
LEFT JOIN dw.dim_customer     c  ON c.customer_id   = src.customer_id
                                AND c.is_current     = 1
LEFT JOIN dw.dim_promotion    pr ON pr.promotion_code = src.promotion_code
LEFT JOIN dw.dim_employee     e  ON e.employee_id   = src.employee_id
                                AND e.is_current     = 1

-- Exclude rows that went to rejection table
WHERE src.transaction_number NOT IN
      (SELECT transaction_number FROM stg.stg_sales_rejected);

```

Student

Now load the sales fact - with the DQ gate running first - and handle a late-arriving store.

Claude • AI Assistant

Two ETL patterns in one subsection: the five-rule DQ gate that runs on staging data before any row touches the fact table, and the late-arriving dimension placeholder pattern when a new store's transactions arrive before its `dim_store` row is loaded.

CONCEPT CARD

The Mango Mart DQ gate - five rules on staging:

1. **Null check** - `transaction_number` , `product_code` , `store_code` , `sale_date` , `net_revenue` must not be NULL.
2. **Referential integrity** - every `product_code` in staging must resolve to an active `dim_product` row (`iscurrent=1`). Every `storecode` must resolve to `dim_store`.
3. **Range check** - `net_revenue` between 0 and 500,000. `quantity_sold` between 1 and 1000. Negative or absurdly large values go to the rejection table.
4. **Volume anomaly** - today's staging row count compared to the 30-day daily average (day-of-week adjusted). Alert if Z-score > 3 or if count = 0.
5. **Freshness** - max `source_updated_at` in staging must be within the last 6 hours. Older = pipeline may be broken.

CONCEPT CARD

Late-arriving store - the placeholder pattern:

1. **Scenario:** Mango Mart opened a new Express store in Kochi on November 1.

The store's first 200 transactions arrive in the ETL staging table on November 1 at 6 AM.

The master data team won't provision the `dim_store` row until November 1 at 11 AM (their batch runs at 10 AM).

2. **Wrong approach:** Hold the 200 fact rows in staging until the `dim_store` row is available.

Risk: if the dimension never arrives (data entry error), the facts are stuck in staging permanently - invisible to the warehouse.

3. **Right approach:** Create a placeholder `dim_store` row immediately when the RI check fails.

Load the fact rows with the placeholder surrogate key. Set `is_placeholder=1`.

When the real `dim_store` arrives at 11 AM, back-update the placeholder row in-place (Type 1). The 200 fact rows automatically resolve to the real store attributes.

EXAMPLE CARD

The DQ gate output for a realistic staging load:

1. 50,000 rows arrive in `stg.stg_sales_source`.
2. Null check: 3 rows fail (NULL `net_revenue` - likely POS system bug). → 3 rows to rejection table.
3. RI check product: 0 failures.
4. RI check store: 200 rows fail (new Kochi store, *storecode not in dimstore*) → placeholder created for store code 'KCH-001', 200 rows loaded with placeholder key.
5. Range check: 1 row fails (`net_revenue = -999`, return entered in wrong system) → rejection table.
6. Volume: 50,200 rows expected, 50,000 arrived → Z-score = 1.4 → WARN but load.
7. **Result:** 49,796 rows loaded to fact, 4 rows in rejection table, 1 placeholder created.

⚠ WARNING

A DQ gate that silently drops rows - without writing them to a rejection table with the rejection reason - is worse than no DQ gate. You lose data AND lose the ability to investigate why. Every rejected row must be written somewhere inspectable.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: Running DQ checks AFTER loading to the fact table. Now bad rows are mixed with good rows and require negating entries to remove. Run DQ BEFORE any production load.

Correct Way: All five DQ rules run on staging data. Clean rows load to fact. Bad rows go to `stg.stg_sales_rejected`. No bad row ever enters the warehouse.

EXAM TIP CARD

Interview classic: *"A new store opened today and sent 500 transactions to your warehouse before the store master record was created in the dimension table. How do you handle this?"*

Winning answer: *"Create an inferred/placeholder dimstore row immediately - storecode filled in, all descriptive attributes set to 'Pending'.*

Load the 500 fact rows with this placeholder surrogate key. Flag the placeholder with `is_placeholder=1`.

When the real store record arrives, back-update the placeholder row in-place. The 500 fact rows automatically resolve correctly.

Never hold facts in staging indefinitely - create the placeholder and load."*

🔥 EXERCISE

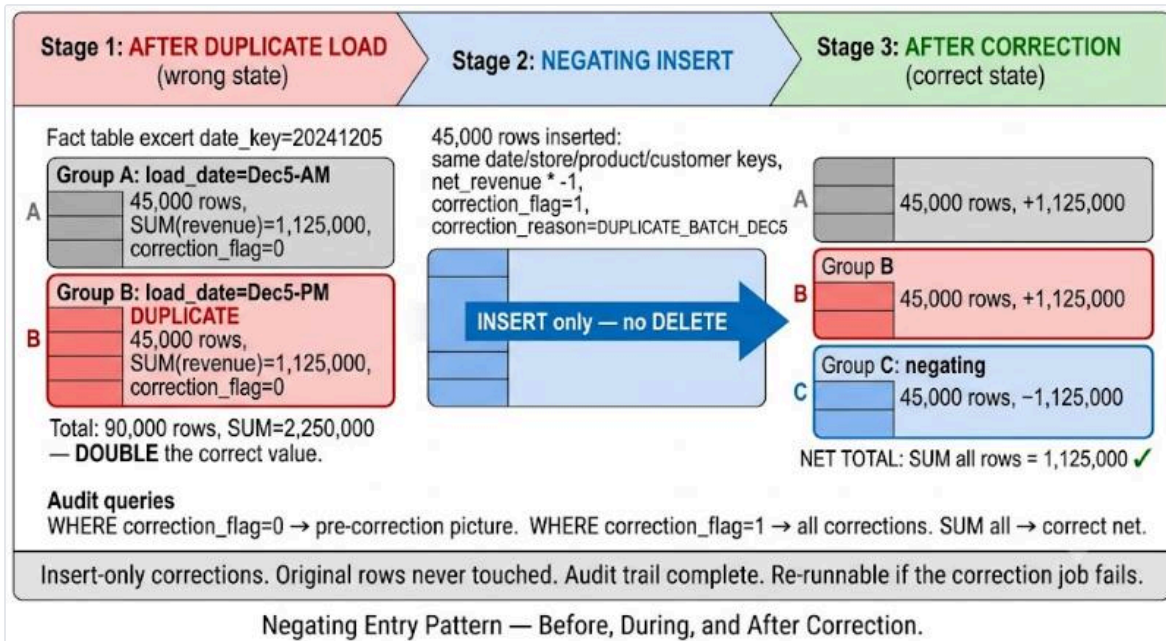
Write the monitoring query that shows: (a) all current placeholders in *dimstore*, (b) *how many factsales* rows reference each placeholder, (c) how many days old each placeholder is. Alert if any placeholder is older than 2 business days.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Design a complete five-rule DQ validation framework for a retail ETL pipeline. For each rule (null check, referential integrity, range check, volume anomaly, freshness), write the SQL, define the WARN vs FAIL threshold, and show the rejection table INSERT that captures failed rows with their rejection reason."*

LAB PART 2-C — FACT CORRECTION WITH NEGATING ENTRY

Claude • AI Assistant



Negating Entry Pattern - Before, During, and After Correction

sql – Negating entry for duplicate batch – detect, correct, verify

```
-- =====
-- STEP 1: DETECT – confirm the duplicate load
-- =====
SELECT
    load_date,
    COUNT(*)          AS row_count,
    SUM(net_revenue)  AS total_revenue
FROM   dw.fact_sales_transaction
WHERE  date_key      = 20241205
      AND correction_flag = 0
GROUP BY load_date
ORDER BY load_date;
-- Shows TWO rows with load_date='2024-12-05' – confirms duplicate batch
-- OR double the expected row count for that date

-- =====
-- STEP 2: NEGATING INSERT – cancels the duplicate batch
-- One negation row per duplicate original row
-- =====
INSERT INTO dw.fact_sales_transaction (
    date_key, store_key, product_key, customer_key,
    promotion_key, employee_key, transaction_number,
    quantity_sold, unit_price, net_revenue, tax_amount,
    load_date, correction_flag, correction_reason
)
```

```

SELECT
    date_key, store_key, product_key, customer_key,
    promotion_key, employee_key, transaction_number,
    -quantity_sold,      -- NEGATED
    unit_price,          -- not negated (non-additive, not summed)
    -net_revenue,        -- NEGATED
    -tax_amount,         -- NEGATED
    CURRENT_DATE,
    1,                  -- correction_flag
    'DUPLICATE_BATCH_DEC5'
FROM   dw.fact_sales_transaction
WHERE  date_key         = 20241205
      AND load_date      = '2024-12-05'  -- the duplicate load date
      AND correction_flag = 0             -- original rows only
      -- Idempotency: don't re-run if already corrected
      AND NOT EXISTS (
        SELECT 1 FROM dw.fact_sales_transaction
        WHERE  correction_reason = 'DUPLICATE_BATCH_DEC5'
        LIMIT 1
      );

-- =====
-- STEP 3: VERIFY – net revenue should now equal correct total
-- =====

SELECT
    SUM(net_revenue) AS net_total,
    SUM(CASE WHEN correction_flag=0 THEN net_revenue END) AS
before_correction,
    SUM(CASE WHEN correction_flag=1 THEN net_revenue END) AS
correction_amount,
    COUNT(*) FILTER (WHERE correction_flag=1) AS correction_rows
FROM   dw.fact_sales_transaction
WHERE  date_key = 20241205;
-- net_total should = before_correction / 2 (half the erroneously doubled
total)

```

Student

The ETL job ran twice on December 5 - all rows are doubled. How do I fix it without destroying the audit trail?

The negating entry pattern from Chapter 4 applied to a real scenario. The principle: never DELETE original rows from a fact table.

Insert negating rows (same dimension keys, negative measures) to cancel the error. Then if needed, insert correct rows.

Net total is correct; audit trail is complete.

CONCEPT CARD

The duplicate batch scenario:

1. **What happened:** The ETL job for December 5 was triggered twice (a scheduling bug). 45,000 rows loaded twice. `fact_sales_transaction` has 90,000 rows for `date_key = 20241205`.
2. **Detection:** `SELECT load_date, COUNT(*) FROM fact_sales_transaction WHERE date_key = 20241205 AND correction_flag = 0 GROUP BY load_date` shows two load dates for December 5, or double the expected row count.
3. **Fix:** Insert 45,000 negating rows - same dimension keys, `quantity_sold * -1`, `net_revenue * -1`, `correction_flag = 1`, `correction_reason = 'DUPLICATE_BATCH_DEC5'`.
4. **Net effect:** `SUM(net_revenue) WHERE date_key = 20241205` = original + duplicate + negation = 1× correct total.
5. **Audit:** `WHERE correction_flag = 0` shows the pre-correction picture. `WHERE correction_flag = 1` shows all corrections applied. SUM of all rows = correct net total.

CONCEPT CARD

Three correction scenarios and their patterns:

1. **Duplicate batch** - insert negating rows for all duplicated rows. Do not re-insert correct rows (they're already there - the original load was correct).
2. **Wrong measure value** - insert one negating row + one correct row per affected transaction.
3. **Wrong dimension key** - insert one negating row (with the wrong key) + one correct row (with the correct key). This corrects the dimensional attribution, not just the measure.

EXAMPLE CARD

Verification after correction:

sql – Code Example

```
-- Pre-correction revenue (original rows only)
SELECT SUM(net_revenue) AS before_correction
```

```

FROM fact_sales_transaction
WHERE date_key = 20241205 AND correction_flag = 0;
-- Result: 2,250,000 (double the real number)

-- Net revenue after corrections (all rows including negation)
SELECT SUM(net_revenue) AS after_correction
FROM fact_sales_transaction
WHERE date_key = 20241205;
-- Result: 1,125,000 (correct)

-- Correction audit
SELECT correction_reason, COUNT(*) AS correction_rows,
       SUM(net_revenue) AS net_correction_amount
FROM fact_sales_transaction
WHERE correction_flag = 1 AND date_key = 20241205
GROUP BY correction_reason;
-- Shows: DUPLICATE_BATCH_DEC5 | 45,000 | -1,125,000

```

🔗 ANALOGY

Negating entries are the **accounting journal's correcting entry**. In double-entry bookkeeping, you never erase a wrong entry - you add a reversing entry and a correcting entry.

The ledger shows all three. Auditors can trace every correction to its original transaction and reason.

This is why financial systems and dimensional warehouses both use this pattern.

⚠️ WARNING

`DELETE FROM fact_sales_transaction WHERE date_key = 20241205 AND load_date = '2024-12-05'` is tempting and catastrophically wrong. (a) Downstream systems may have already consumed these rows.

(b) The audit trail is destroyed. (c) In partitioned systems, DELETE is expensive.

(d) If the re-load also has a bug, there is no record of what was there before.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Truncating and reloading the partition. Same problems as DELETE - destroys audit trail, risky if re-load fails.

Correct Way: Negating entries. Insert-only. The original rows remain. Corrections are visible and dated. Net total is correct. Re-runnable if the correction job itself fails (idempotent with the RIGHT correction_reason filter).

EXAM TIP CARD

Interview classic: "How do you fix a fact table where a batch was loaded twice?" Winning answer: *"Negating entry pattern. INSERT one negation row per duplicate row: same dimension keys, negative measures, `correctionflag=1`, `correctionreason='DUPLICATE_BATCH'`."*

Do not re-insert correct rows - the original load was correct, only the duplicate needs negation. Verify: `SUM(net_revenue)` across all rows (original + duplicate + negation) should equal the correct total.

The audit trail is preserved."

EXERCISE

A decimal error loaded `net_revenue = 4500.00` when the correct value is `450.00` for transaction TXN-2024-09876. Write: (a) the negating INSERT, (b) the correcting INSERT, (c) the verification query showing the net revenue before and after correction for this transaction.

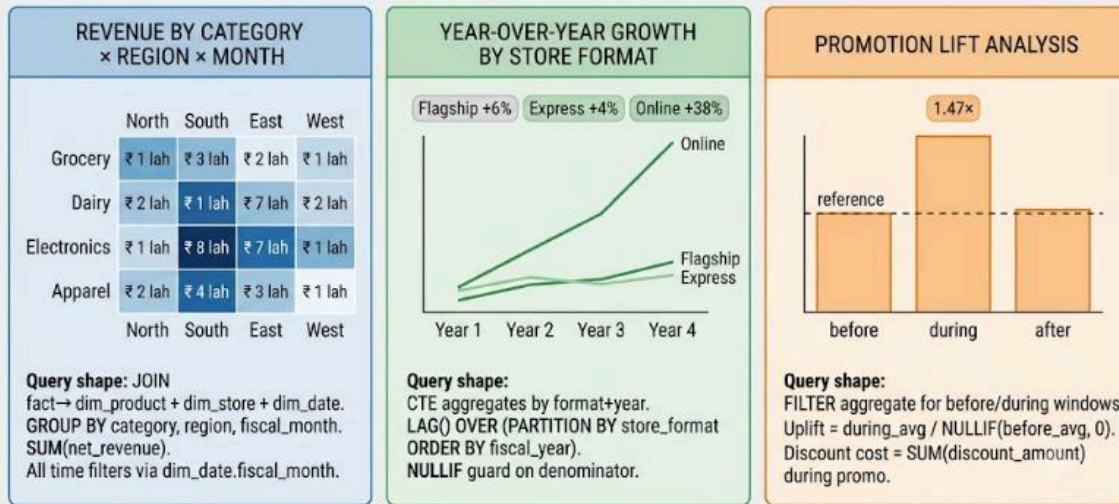
AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: *"Design a complete fact correction management system. Include: a `corrections_log` table that records every correction event, the negating entry INSERT template for three error types (duplicate batch, wrong measure, wrong dimension key), a correction audit report query, and an idempotency check that prevents the same correction from being applied twice."*

LAB PART 3-A — SALES PERFORMANCE ANALYTICS

 Claude • AI Assistant

Sales Analytics Query Patterns — Three Queries, Three Business Questions.



Sales Analytics Query Patterns - Three Queries, Three Business Questions

sql – Three Mango Mart sales analytics queries

```
-- =====
-- QUERY 1: Revenue by category × region × fiscal month
-- =====

SELECT
    p.category_name,
    s.region,
    d.fiscal_year,
    d.fiscal_month,
    SUM(f.net_revenue)          AS net_revenue,
    SUM(f.quantity_sold)       AS units_sold,
    COUNT(DISTINCT f.transaction_number) AS transactions
FROM   dw.fact_sales_transaction f
JOIN   dw.dim_product           p ON f.product_key = p.product_key
JOIN   dw.dim_store             s ON f.store_key   = s.store_key
JOIN   dw.dim_date              d ON f.date_key    = d.date_key
WHERE  d.fiscal_year            = 2025
      AND f.correction_flag     = 0
      AND p.is_current          = 1
GROUP BY p.category_name, s.region, d.fiscal_year, d.fiscal_month
ORDER BY d.fiscal_month, p.category_name, net_revenue DESC;

-- =====
-- QUERY 2: Year-over-year revenue growth by store format (LAG)
-- =====

WITH yearly AS (
```

```

SELECT
    s.store_format,
    d.fiscal_year,
    SUM(f.net_revenue) AS revenue
FROM dw.fact_sales_transaction f
JOIN dw.dim_store s ON f.store_key = s.store_key
JOIN dw.dim_date d ON f.date_key = d.date_key
WHERE f.correction_flag = 0
GROUP BY s.store_format, d.fiscal_year
)
SELECT
    store_format,
    fiscal_year,
    revenue,
    LAG(revenue) OVER (
        PARTITION BY store_format ORDER BY fiscal_year
    ) AS prior_year_revenue,
    ROUND(
        100.0 * (revenue
            - LAG(revenue) OVER (PARTITION BY store_format ORDER BY
fiscal_year))
        / NULLIF(LAG(revenue) OVER (PARTITION BY store_format ORDER BY
fiscal_year), 0)
        , 1) AS yoy_growth_pct
FROM yearly
ORDER BY store_format, fiscal_year;

-- =====
-- QUERY 3: Promotion lift – revenue before / during / after
-- =====
WITH promo_window AS (
    SELECT
        pr.promotion_name,
        pr.campaign,
        pr.start_date,
        pr.end_date,
        d.calendar_date,
        SUM(f.net_revenue) AS daily_revenue,
        SUM(f.discount_amount) AS daily_discount
    FROM dw.fact_sales_transaction f
    JOIN dw.dim_promotion pr ON f.promotion_key =
pr.promotion_key
    JOIN dw.dim_date d ON f.date_key = d.date_key
    WHERE d.calendar_date BETWEEN pr.start_date - INTERVAL '7 days'
        AND pr.end_date + INTERVAL '7 days'
        AND f.correction_flag = 0

```

```

        GROUP BY pr.promotion_name, pr.campaign, pr.start_date, pr.end_date,
        d.calendar_date
    )
    SELECT
        promotion_name, campaign,
        ROUND(AVG(daily_revenue) FILTER (WHERE calendar_date < start_date), 2)
    AS avg_before,
        ROUND(AVG(daily_revenue) FILTER (WHERE calendar_date BETWEEN start_date
    AND end_date), 2) AS avg_during,
        ROUND(
            AVG(daily_revenue) FILTER (WHERE calendar_date BETWEEN start_date AND
    end_date)
            / NULLIF(AVG(daily_revenue) FILTER (WHERE calendar_date <
    start_date), 0)
        , 2)
    AS uplift_ratio,
        ROUND(SUM(daily_discount) FILTER (WHERE calendar_date BETWEEN start_date
    AND end_date), 2) AS discount_cost
    FROM   promo_window
    GROUP BY promotion_name, campaign
    ORDER BY uplift_ratio DESC NULLS LAST;

```

Student

The warehouse is loaded. Now run the analytics queries - starting with sales performance.

Claude • AI Assistant

Three production-quality sales analytics queries: revenue by category and region over fiscal months, year-over-year growth by store format using LAG(), and promotion lift analysis comparing revenue during and outside promotion periods. Every query uses the star schema exactly as designed - two to four joins, GROUP BY dimension attributes, SUM/LAG on additive measures.

CONCEPT CARD

Query design principles applied in this section:

1. **All time filters go through dim_date** - never EXTRACT or DATEPART on the fact table's raw date column. This ensures partition pruning and uses the pre-computed fiscal calendar.

2. **SUM on additive measures only** - *netrevenue*, *quantitysold*, *discountamount*. *Never SUM(unitprice)*.
3. **NULLIF in denominator** - every percentage and ratio uses `NULLIF(denominator, 0)` to prevent division-by-zero.
4. **correction_flag = 0** - standard analytics filters to original rows. Corrections net to zero and would distort comparisons if not excluded from some specific analyses.
5. **is_current = 1** on Type 2 dimensions for current-state reporting - omit when historical accuracy is needed.

EXAMPLE CARD

Revenue by category, region, fiscal month - reading the output:

1. Row: Electronics | South | FY2025 | Q2 | October | ₹8,240,000 | 12,450 units.
2. Meaning: In October of fiscal year 2025 (which started April 2024), Mango Mart's Electronics category generated ₹8.24M in net revenue across all South-region stores, selling 12,450 units.
3. Drill-up: remove `fiscal_month` from GROUP BY → quarterly view. Remove `category_name` → all-category region view. The dimensional model enables arbitrary drill-up without query redesign.
4. Drill-down: add `p.subcategory_name` to SELECT and GROUP BY → subcategory detail within each category. One column addition, no schema change.

🔗 ANALOGY

Analytics over a well-designed star schema is like reading a book with a well-built index. You don't rearrange the book to find a chapter - you consult the index (dimension attributes) and jump directly to the relevant pages (fact rows).

A poorly designed model is a book with no index - you scan every page for every question.

⚠️ WARNING

The YoY growth query uses `LAG()` with `PARTITION BY storeformat ORDER BY fiscalyear`. If a store format has no sales in a particular fiscal year, `LAG()` returns NULL for that partition boundary - the `NULLIF` guard prevents a divide-by-zero crash but produces a NULL growth rate, not zero.

Zero and NULL mean different things: zero means growth measured and found to be zero; NULL means no prior year to compare.

🚫 COMMON PITFALL / MISTAKE

Incorrect Way: `WHERE d.fiscal_year = 2024` in the YoY query instead of filtering to `IN (2023, 2024)`. Then `LAG()` has only one year of data and returns NULL for all prior-year comparisons.

Correct Way: Include both the current AND the prior year(s) in the WHERE clause, or use `relative_period_month >= -23` (24 months of data) and let LAG() find the natural prior-year row.

EXAM TIP CARD

Interview classic: "Write a query showing year-over-year revenue growth by store format."

Winning answer: *CTE: aggregate revenue by storeformat and fiscalyear. Outer query: LAG(revenue) OVER (PARTITION BY storeformat ORDER BY fiscalyear) for prior-year value.

Growth = (current - prior) / NULLIF(prior, 0) × 100. Include at least two years of data in the WHERE clause so LAG() has a prior row to reference.*

🔥 EXERCISE

Modify the category-region-month query to add a column showing each category's revenue as a % of the total revenue for that region and month (market share within region). Use a window function: `SUM(revenue) OVER (PARTITION BY region, fiscal_year, fiscal_month)` as the denominator.

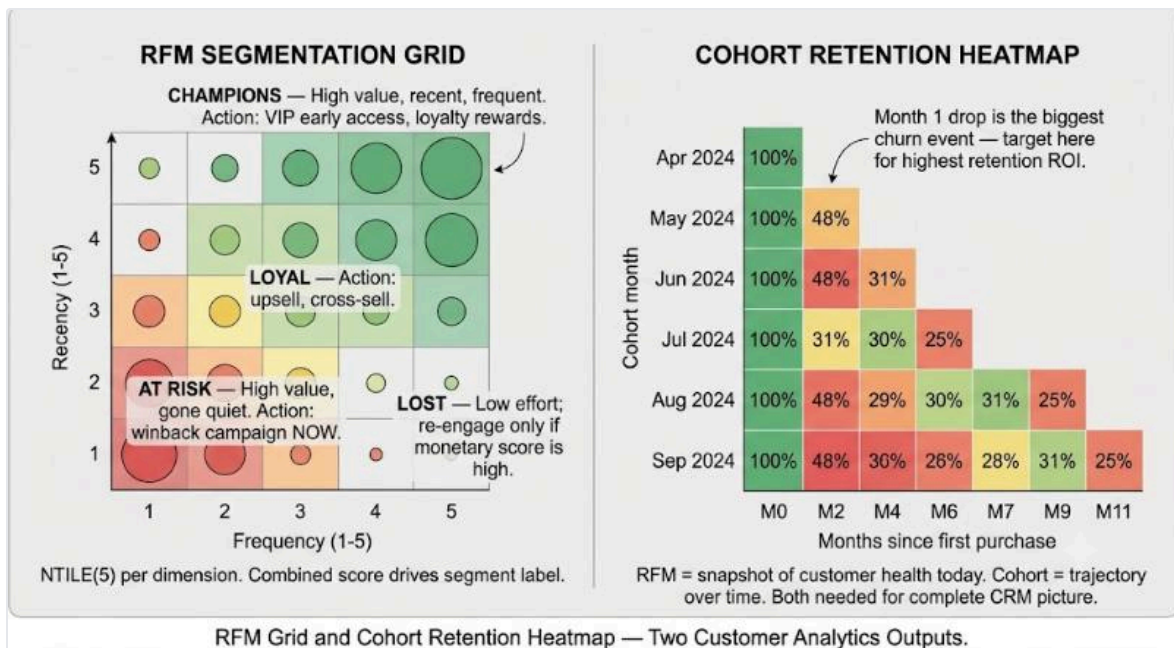
🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "For a retail star schema, write a promotion performance query that shows: (a) revenue per store-day during the promotion, (b) revenue per store-day in the 7 days before the promotion, (c) uplift ratio (during / before), (d) total discount cost, and (e) net uplift dollars. Include the FILTER aggregate syntax."

LAB PART 3-B — CUSTOMER ANALYTICS: RFM AND COHORT RETENTION



Claude • AI Assistant



RFM Grid and Cohort Retention Heatmap - Two Customer Analytics Outputs

sql – RFM segmentation and cohort retention for Mango Mart

```
-- =====
-- QUERY 4: RFM Segmentation with NTILE scoring and labels
-- =====

WITH rfm_base AS (
    SELECT
        f.customer_key,
        MAX(f.date_key)                                AS last_purchase_key,
        COUNT(DISTINCT f.transaction_number)           AS frequency,
        SUM(f.net_revenue)                             AS monetary
    FROM   dw.fact_sales_transaction f
    WHERE  f.customer_key <> -1      -- exclude Unknown Customer
        AND f.correction_flag = 0
    GROUP BY f.customer_key
),
rfm_scored AS (
    SELECT
        customer_key, last_purchase_key, frequency, monetary,
        NTILE(5) OVER (ORDER BY last_purchase_key DESC) AS r, -- 5=most
        recent
        NTILE(5) OVER (ORDER BY frequency DESC)           AS f, -- 5=most
        frequent
        NTILE(5) OVER (ORDER BY monetary DESC)             AS m  -- 5=highest
        spend
    FROM   rfm_base
)
```

```

SELECT
    rs.customer_key,
    c.customer_name,
    c.email,
    c.loyalty_tier,
    rs.monetary,
    rs.frequency,
    rs.r, rs.f, rs.m,
    CASE
        WHEN rs.r >= 4 AND rs.f >= 4 THEN 'Champions'
        WHEN rs.r >= 3 AND rs.f >= 3 THEN 'Loyal'
        WHEN rs.r >= 4 AND rs.f < 3 THEN 'Promising'
        WHEN rs.r < 3 AND rs.f >= 4 THEN 'At Risk'
        WHEN rs.r < 2 AND rs.f < 2 THEN 'Lost'
        ELSE 'Needs Attention'
    END AS rfm_segment
FROM   rfm_scored      rs
JOIN   dw.dim_customer c ON rs.customer_key = c.customer_key
                        AND c.is_current = 1
ORDER BY rs.monetary DESC;

-- =====
-- QUERY 5: Monthly cohort retention curve
-- =====

WITH first_orders AS (
    SELECT
        f.customer_key,
        MIN(d.date_key) AS first_purchase_key,
        MIN(d.fiscal_year) * 100 + MIN(d.fiscal_month) AS cohort_ym
    FROM   dw.fact_sales_transaction f
    JOIN   dw.dim_date d ON f.date_key = d.date_key
    WHERE  f.customer_key <> -1 AND f.correction_flag = 0
    GROUP BY f.customer_key
),
orders_with_cohort AS (
    SELECT
        fo.cohort_ym,
        f.customer_key,
        (d.fiscal_year * 100 + d.fiscal_month) - fo.cohort_ym AS months_since
    FROM   dw.fact_sales_transaction f
    JOIN   dw.dim_date d ON f.date_key = d.date_key
    JOIN   first_orders fo ON f.customer_key = fo.customer_key
    WHERE  f.customer_key <> -1 AND f.correction_flag = 0
),
cohort_sizes AS (
    SELECT cohort_ym, COUNT(*) AS cohort_size

```

```

        FROM    first_orders
        GROUP   BY cohort_ym
    )
    SELECT
        o.cohort_ym,
        c.cohort_size,
        o.months_since,
        COUNT(DISTINCT o.customer_key)                AS
active_customers,
        ROUND(100.0 * COUNT(DISTINCT o.customer_key)
              / NULLIF(c.cohort_size, 0), 1)          AS
retention_pct
    FROM    orders_with_cohort o
    JOIN    cohort_sizes      c ON c.cohort_ym = o.cohort_ym
    WHERE   o.months_since <= 11
    GROUP   BY o.cohort_ym, c.cohort_size, o.months_since
    ORDER   BY o.cohort_ym, o.months_since;

```

Student

Now run the customer analytics - RFM segmentation and cohort retention - using the actual dim_customer SCD Type 2 dimension correctly.

Claude • AI Assistant

Two advanced customer analytics patterns. RFM segmentation uses NTILE() to score each customer on Recency, Frequency, and Monetary value - then labels segments by combined score.

Cohort retention uses a three-CTE pattern to identify each customer's first-purchase cohort month and track the % active in each subsequent month.

CONCEPT CARD

Using SCD Type 2 correctly in analytics:

1. **Current-state analytics** (RFM, LTV) - filter `is_current = 1` on dim_customer. You want the customer's CURRENT loyalty tier and segment for targeting.
2. **Historical accuracy analytics** (what tier was the customer when they bought?) - join fact.customerkey directly to dimcustomer.customerkey *WITHOUT filtering iscurrent*.

The fact row's customer_key already points to the correct version via the surrogate key assigned at load time.

3. **Cross-time analysis** - compare RFM behavior across acquisition cohorts by using `dimcustomer.firstpurchase_dt` as the cohort anchor.
4. **Unknown Customer exclusion** - always filter `WHERE f.customer_key <> -1` in any customer-level analysis. The unknown customer surrogate aggregates all anonymous transactions as one fictional customer.

CONCEPT CARD

RFM scoring - the NTILE pattern:

1. **Recency** - `MAX(date_key)` per customer. Higher = more recent = better. `NTILE(5) OVER (ORDER BY last_purchase DESC)` → score 5 = most recent.
2. **Frequency** - `COUNT(DISTINCT transaction_number)` per customer. Higher = more frequent = better. `NTILE(5) OVER (ORDER BY freq DESC)` → score 5 = most frequent.
3. **Monetary** - `SUM(net_revenue)` per customer. Higher = more spend = better. `NTILE(5) OVER (ORDER BY total_spend DESC)` → score 5 = highest spender.
4. **Segment labels** - Champions: $R \geq 4$ AND $F \geq 4$. At Risk: $R \leq 2$ AND $F \geq 4$. Lost: $R \leq 2$ AND $F \leq 2$. Promising: $R \geq 4$ AND $F \leq 2$. Others: middle.

CONCEPT CARD

Cohort retention - the three-CTE pattern:

1. **CTE 1: first_orders** - for each customer, compute the fiscal month of their FIRST purchase (`MIN date_key`). This is the cohort assignment.
2. **CTE 2: orders_with_cohort** - tag every purchase with its customer's cohort month and compute `monthssincecohort` as an integer offset.
3. **CTE 3: cohort_sizes** - `COUNT(DISTINCT customer_key)` per cohort month - the denominator for retention %.
4. **Final query** - `COUNT(DISTINCT customer_key)` per cohort × offset, divided by cohort size = retention rate.

⚠ WARNING

Use `first_purchase_dt` from `dimcustomer` for cohort assignment if available - it's already computed and avoids re-deriving `MIN(datekey)` from the fact table. But if the customer dimension is Type 2, `first_purchase_dt` is a stable attribute (Type 1 - never changes) and appears correctly on ALL versions of the customer row.

EXAM TIP CARD

Interview classic: "How do you compute cohort retention in SQL?" Winning answer: "Three CTEs: `firstorders` (`MIN(datekey)` per customer → cohort month), `orderswithcohort` (tag every order with cohort + months-since-cohort offset), `cohortsizes` (`COUNT` per cohort for denominator). Final: `COUNT(distinct customers per cohort × offset) / cohortsize × 100` = retention %.

Filter `customerkey ≠ -1 (unknown)`. Filter `cohortmonth` to last 12 months to keep the result triangular and manageable."

🔥 EXERCISE

Extend the RFM query to add a fourth dimension: tenure (T) = months since `first_purchase_dt`. Champions with long tenure are 'Loyalists' (protect them). Champions with short tenure are 'New Champions' (develop them). Add a segment sub-label based on tenure NTILE.

🌟 AI DEEP DIVE PROMPT

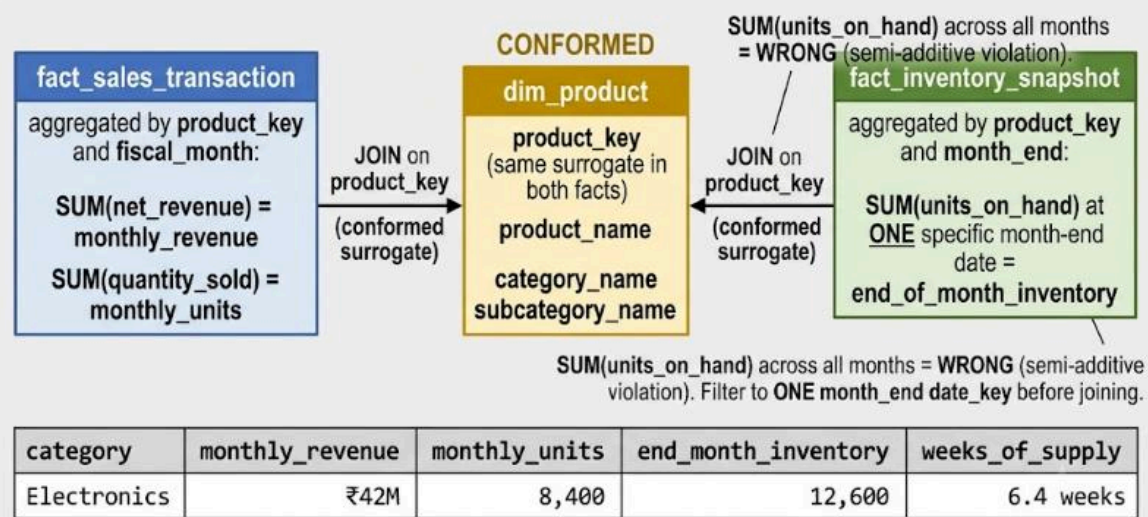
Paste into ChatGPT or Claude: `""Extend the Mango Mart RFM segmentation to produce an actionable marketing output. For each RFM segment, recommend: (a) the right marketing message, (b) the right channel (email, SMS, push), (c) the appropriate offer type (discount, loyalty points, exclusive access).`

Then show the SQL that produces this output alongside the RFM scores.""

LAB PART 3-C — INVENTORY ANALYTICS AND CROSS-FACT DRILL-ACROSS

Claude • AI Assistant

Caption: Cross-Fact Drill-Across — Conformed dim_product Connects Sales and Inventory



This query works ONLY because both facts share the same `dim_product` surrogate key sequence. This is the Bus Matrix payoff.

Cross-Fact Drill-Across - Conformed dim_product Connects Sales and Inventory

sql – Inventory analytics and cross-fact drill-across – weeks of supply

```
-- =====
-- QUERY 6: Inventory at one point in time (semi-additive correct)
-- =====
SELECT
    p.category_name,
    s.region,
    SUM(i.units_on_hand) AS total_units_on_hand, -- SUM across stores: OK
    (not time)
    SUM(i.units_on_order) AS total_units_on_order, -- SEMI-ADD: correct for
one period
    MIN(i.units_on_hand) AS min_store_inventory,
    MAX(i.units_on_hand) AS max_store_inventory
FROM dw.fact_inventory_snapshot i
JOIN dw.dim_product p ON i.product_key = p.product_key AND
p.is_current = 1
JOIN dw.dim_store s ON i.store_key = s.store_key
WHERE i.month_end_date_key = 20241031 -- ONE specific month-end only
GROUP BY p.category_name, s.region
ORDER BY p.category_name, s.region;

-- =====
-- QUERY 7: 3-month average inventory trend (correct time aggregation)
-- =====
SELECT
    p.category_name,
    d.fiscal_month,
    d.fiscal_year,
    AVG(i.units_on_hand) AS avg_monthly_inventory -- AVG across time:
correct
FROM dw.fact_inventory_snapshot i
JOIN dw.dim_product p ON i.product_key = p.product_key AND
p.is_current = 1
JOIN dw.dim_date d ON i.month_end_date_key = d.date_key
WHERE d.relative_period_month BETWEEN -2 AND 0 -- last 3 fiscal months
GROUP BY p.category_name, d.fiscal_month, d.fiscal_year
ORDER BY p.category_name, d.fiscal_year, d.fiscal_month;

-- =====
-- QUERY 8: CROSS-FACT DRILL-ACROSS – weeks of supply
-- Requires conformed dim_product (same product_key in both facts)
-- =====
WITH sales_summary AS (
    -- Aggregate fact_sales for last complete fiscal month
    SELECT
        f.product_key,
```

```

        SUM(f.quantity_sold) AS monthly_units_sold,
        SUM(f.net_revenue)   AS monthly_revenue
FROM   dw.fact_sales_transaction f
JOIN   dw.dim_date d ON f.date_key = d.date_key
WHERE  d.relative_period_month = -1  -- last complete fiscal month
      AND f.correction_flag = 0
GROUP BY f.product_key
),
inventory_summary AS (
    -- Aggregate fact_inventory at last month-end
    SELECT
        i.product_key,
        SUM(i.units_on_hand) AS end_month_inventory
    FROM   dw.fact_inventory_snapshot i
    JOIN   dw.dim_date d ON i.month_end_date_key = d.date_key
    WHERE  d.relative_period_month = -1  -- same month-end
    GROUP BY i.product_key
)
-- Join the two aggregates via conformed dim_product key
SELECT
    p.category_name,
    p.product_name,
    s.monthly_revenue,
    s.monthly_units_sold,
    inv.end_month_inventory,
    -- Weeks of supply: inventory / weekly sales rate (monthly/4.3)
    ROUND(
        inv.end_month_inventory
        / NULLIF(s.monthly_units_sold / 4.3, 0)
        , 1) AS weeks_of_supply,
    CASE
        WHEN inv.end_month_inventory / NULLIF(s.monthly_units_sold / 4.3, 0)
        < 2
            THEN 'REORDER ALERT'
        WHEN inv.end_month_inventory / NULLIF(s.monthly_units_sold / 4.3, 0)
        > 16
            THEN 'OVERSTOCK ALERT'
        ELSE 'HEALTHY'
    END AS inventory_status
FROM   sales_summary s
JOIN   inventory_summary inv ON inv.product_key = s.product_key --
CONFORMED KEY
JOIN   dw.dim_product p ON s.product_key = p.product_key AND
p.is_current = 1
ORDER BY weeks_of_supply ASC NULLS LAST;

```

Final analytics - inventory queries and the cross-fact drill-across that only works because of conformed dimensions.

Claude • AI Assistant

Two patterns that demonstrate the payoff of the Bus Matrix: semi-additive inventory queries (always the most mishandled query type) and the cross-fact drill-across that joins `fact_sales_transaction` to `fact_inventory_snapshot` via `dim_product` - only possible because both facts use the same conformed `product_key` from the same master `dim_product`.

CONCEPT CARD

Semi-additive inventory queries - the rules:

1. **SUM across stores at one point in time** - correct. `SUM(units_on_hand) GROUP BY store_key WHERE month_end_date_key = 20241031` = total inventory at each store at October month-end.
2. **SUM across products at one point in time** - correct. Summation across the product dimension is meaningful.
3. **SUM across time periods** - WRONG. `SUM(units_on_hand)` for Jan + Feb + Mar = meaningless accumulation of month-end snapshots.
4. **Correct time aggregation** - AVG (average monthly inventory level), MAX (peak inventory), MIN (trough), or LAST (most recent). Never SUM across time.
5. **Out-of-stock detection** - rows where `units_on_hand = 0` even though `units_on_order > 0`. These are the at-risk stockout situations.

CONCEPT CARD

Cross-fact drill-across - the conformed dimension payoff:

1. **Business question** - 'For each product category, what was the monthly revenue AND the end-of-month inventory level? Calculate weeks of supply.'
2. **Why it works** - `fact_sales_transaction` and `fact_inventory_snapshot` BOTH use the same `dim_product.product_key`. A JOIN between their aggregates on `product_key` is valid and meaningful.
3. **Why it wouldn't work with siloed marts** - if each mart had its own `dim_product` with its own key sequence, `product_key = 4421` might be 'Samsung TV' in Sales and 'Milk 500ml' in Inventory. The join would be nonsense.

4. **The drill-across pattern** - aggregate each fact separately in CTEs, then JOIN the CTEs on the conformed dimension key. Never join two fact tables directly.

EXAMPLE CARD

Weeks of supply - the key inventory KPI:

1. **Formula** - $\text{end_of_month_inventory} / (\text{monthly_units_sold} / 4.3) = \text{approximate weeks of supply.}$
2. **Interpretation** - 12 weeks of supply for a fashion item = overstocked (fashion has 8-week season). 2 weeks of supply for a grocery staple = understocked (typical replenishment is weekly).
3. **Action trigger** - any category with < 2 weeks of supply gets a reorder alert. Any category with > 16 weeks of supply gets a markdown recommendation.
4. **Cross-fact dependency** - you cannot compute this without BOTH the inventory snapshot (for *units_onhand*) AND the sales fact (for *units_sold* rate). The conformed dimensions make this join possible.

🔗 ANALOGY

The cross-fact drill-across is like **combining two government surveys that sampled the same population with the same participant ID**. If both surveys use the same unique identifier, you can join them: 'this person answered income=high in the economic survey AND health=poor in the health survey.' If different identifiers were used, the join is invalid. Conformed dimension keys are the shared participant ID that makes cross-fact analysis valid.

⚠️ WARNING

Semi-additive query violations are the most common silent error in inventory analytics. A BI tool that defaults to SUM on all numeric columns will SUM *units_on_hand* across 12 months and show 12× the real inventory.

Always configure BI tools to use LAST NON-EMPTY or AVG for semi-additive measures. Document this in the data dictionary.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: `SELECT p.category_name, SUM(i.units_on_hand) FROM fact_inventory_snapshot i JOIN dim_product p ... GROUP BY p.category_name` - sums inventory across ALL month-end snapshots in the table. The result is meaningless.

Correct Way: Add `WHERE i.month_end_date_key = :specific_month_end` - one point in time. Then SUM across products/stores is correct. For trend: use AVG per month, not SUM.

EXAM TIP CARD

Interview classic: "How do you write a query that combines monthly sales revenue with end-of-month inventory from two separate fact tables?" Winning answer: *"Two CTEs: one aggregates factsales by product and fiscalmonth (SUM - additive); one aggregates factinventorysnapshot by product and monthend date (SUM - correct because it's one time period per row). JOIN the two CTEs on productkey - valid because it's the same conformed surrogate key from the same master dim_product.*

Calculate weeksofsupply as inventoryunits / (monthlyunitssold / 4.3). This drill-across only works because both facts share the conformed dimproduct."

EXERCISE

Write the out-of-stock detection query: find every product × store combination at the most recent month-end where `units_on_hand = 0` AND `units_on_order = 0`. Join to `dimproduct` and `dimstore` to show category and region.

Sort by category to show which categories have the most out-of-stock situations.

AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "For the Mango Mart star schema, write a complete inventory health dashboard query. Include: current stock level by category (last month-end only), 3-month average inventory trend (AVG not SUM), weeks of supply by category using the cross-fact drill-across pattern, out-of-stock alerts (`units_on_hand = 0`), and a return-rate overlay from `fact_customer_returns`."

LAB REVIEW & SUMMARY

Claude • AI Assistant

CHAPTER SUMMARY

You have now built, loaded, corrected, and queried a complete retail dimensional warehouse from scratch. Every concept from Chapters 1 through 5 appeared in this lab - grounded in runnable SQL on a real schema.

- LAB PART 1-A - DESIGN AND BUS MATRIX: Applied the four-step Kimball process to three Mango Mart business processes. Wrote grain statements for each. Built the Bus Matrix - three conformed dimensions (date, store, product) with three checkmarks each. Established the build sequence: conformed dimensions first, fact tables last.

- LAB PART 1-B - DIMENSION DDL: Built six dimension tables. Applied SCD type decisions per attribute - not per table. *dimcustomer* and *dimproduct*: Type 2 on specific tracked attributes. *dimstore*: *Type 1*. *dimpromotion*: Type 0. Created Unknown Customer (key=-1) and Unknown Store (key=-1) surrogate placeholders for optional FKs.
- LAB PART 1-C - FACT TABLE DDL: Built three fact tables at three different grains. Composite PK on *factinventorysnapshot* prevents duplicate month-end loads. Correction metadata (*loaddate*, *correctionflag*, *correctionreason*) *added at creation time*. *promotionkey* nullable in *fact_sales* - not every transaction has a promotion.
- LAB PART 2-A - SCD TYPE 2 LOAD: Executed the two-step MERGE: Step 1 expires changed rows (UPDATE *iscurrent*=0, *rowend*=yesterday). Step 2 inserts new versions. Traced the five *dim_customer* rows produced from three source records after the Day 30 load. Ran the consistency check (0 rows expected).
- LAB PART 2-B - DQ GATE AND LATE-ARRIVING DIMENSION: Applied five DQ rules to staging data. 4 rows rejected (3 null, 1 out-of-range). 200 rows had unresolvable *storecode* → *placeholder dimstore* created with *is_pending*=1. 49,796 clean rows loaded. Surrogate key lookup via JOIN to dim tables resolved all natural keys to integer FKs.
- LAB PART 2-C - FACT CORRECTION: Detected duplicate batch by checking *loaddate* row counts. *Applied negating entry pattern: 45,000 negation rows inserted with correctionflag=1 and correctionreason='DUPLICATEBATCH_DEC5'*. Verified: net total = correct single-load total. Audit trail preserved - original rows untouched.
- LAB PART 3-A - SALES ANALYTICS: Three queries: revenue by category × region × fiscal month (2-3 joins + GROUP BY), YoY growth by store format (LAG() OVER PARTITION BY), promotion lift (FILTER aggregate for before/during windows). All time filters via *dim_date*; all aggregations on additive measures only.
- LAB PART 3-B - CUSTOMER ANALYTICS: RFM segmentation with NTILE(5) per dimension and CASE-based segment labels. Cohort retention with three-CTE pattern (*firstorders* → *orderswithcohort* → *cohortsizes*). *customerkey* ≠ -1 filter on all customer queries. *iscurrent*=1 for current-state; no filter for historical-accuracy analysis.
- LAB PART 3-C - INVENTORY AND CROSS-FACT: Semi-additive queries with WHERE clause restricting to one month-end (never SUM across time). Cross-fact drill-across joining *factsales* and *factinventory* aggregates via conformed *dimproduct* *productkey*. Weeks-of-supply computed as *endmonthinventory* / (monthly_units / 4.3). Inventory alerts for < 2 weeks and > 16 weeks.

Claude • AI Assistant

You have completed the full Dimensional Deep Dive. Grain, measures, SCD types, Bus Matrix, MERGE patterns, DQ gates, correction entries, RFM, cohort retention, semi-additive

queries, cross-fact drill-across - every concept has been both explained and implemented in runnable SQL.

The Mango Mart schema you built in this lab is the template: same structure, different domain, every production warehouse you will ever design. Go build your own.